

The Trajectory of Code

Survival-Weighted Learning from Version-Control History for Agentic Software Models

Zach Kelling

Hanzo Industries Inc (Techstars '17)

zach@hanzo.ai ORCID: — @zeekay

HAI-2026-014 • Version 0.1 (Draft) • INTERNAL / PRIVATE

Abstract

We argue that the most valuable training signal for agentic software models is not the agent’s transcript nor the final source tree, but the *longitudinal history of the code itself*. A version-control graph is a record of natural selection: every line, hunk, and structural *shape* that an author commits is subsequently either preserved, rewritten, or deleted by the future. Survival therefore constitutes a time-validated, annotation-free quality label at massive scale. We formalize *code survival* S (the duration a unit of code persists before it is changed or removed), extend it from lines to abstract structural shapes (σ), and show that survival-weighted objectives let a model learn to generate code that *lasts* rather than code that merely parses or passes a single test. We describe a decomplexed extraction pipeline that fuses three co-registered layers—agent *trajectories* ($\sim 5,190$ Claude Code sessions), verified *diffs* (file-history and git), and *outcomes* (survival, test status, and a per-commit HUMAN/AI label)—across a heterogeneous fleet of developer machines, with mandatory secret redaction and provenance versioning. The result is a corpus of the form (*instruction* $\rightarrow$ *agent trajectory* $\rightarrow$ *verified diff* $\rightarrow$ *survival outcome*), which supports supervised, preference, and outcome-verified reinforcement objectives, and a novel evaluation regime—*survival holdout*—that scores a model on whether its generated changes would have endured.

Contents

1	Thesis: time is the annotator	2
2	Formalism: code survival	2
2.1	Line and hunk survival	2
2.2	From lines to shapes	3
2.3	Durability label	3
3	The three co-registered layers	3
3.1	The human/ai axis	4
4	Construction pipeline	4
5	Training and evaluation formulations	4
6	Why survival-weighting produces better agents	5
7	Privacy, provenance, and ethics	5
8	Conclusion	5

1 Thesis: time is the annotator

Public code datasets are overwhelmingly *cross-sectional*: a snapshot of a repository at one instant, scraped and tokenized. Agentic benchmarks add the task dimension (an instruction plus a repository state), and instruction-tuned corpora add human preference labels. All three ask the same implicit question: *what code was written?* None asks the question that a working engineer actually cares about: *what code should have been written—and how do we know?*

A version-control history answers that question for free, because it contains the future. Every unit of code an author commits is, over the following weeks and years, subjected to a merciless empirical test: it is kept, edited, or deleted by real subsequent work. Code that survives dozens of refactors was *validated by time*; code rewritten the next morning is a labeled negative. No annotator, rubric, or reward model produced these labels—the history did, and it did so at the scale of every commit ever made.

This paper develops that observation into a training methodology. Our central claims are:

1. **Survival is a quality signal.** The persistence of a unit of code is a time-validated, annotation-free proxy for its quality, correctness, and architectural fit.
2. **Shapes, not lines, are the unit of interest.** Line survival is noisy; the durable signal lives at the level of *shapes*—function signatures, module boundaries, abstractions, and interfaces. Some shapes last far longer than others, and those are the good bones.
3. **Trajectory beats snapshot.** A model should learn how code *moves*—the ordered sequence of states a file passes through—not merely its terminal form.
4. **The three layers must be co-registered.** Agent reasoning (the HOW), the verified diff (the WHAT), and the survival outcome (the RESULT) are individually useful and jointly transformative: they yield grounded tuples (*instruction* $\rightarrow$ *trajectory* $\rightarrow$ *diff* $\rightarrow$ *outcome*) that no single-layer corpus can express.

2 Formalism: code survival

2.1 Line and hunk survival

Let a repository’s history be an ordered sequence of commits $c_0, c_1, \dots, c_T$. For a unit of code u (a line, a hunk, or a shape) introduced at commit c_b (*birth*) and first modified or removed at commit c_d (*death*), define its survival as the wall-clock duration

$$S(u) = t(c_d) - t(c_b), \quad E(u) = d - b,$$

where $E(u)$ is the number of intervening commits the unit *survived through* (its edit-distance longevity). A unit still present at the head commit c_T is right-censored; we treat it as alive with a lower bound $S(u) \geq t(c_T) - t(c_b)$ and handle censoring with standard survival-analysis estimators (Kaplan–Meier for population curves; Cox models when conditioning on features).

Line survival is recoverable from `git` via incremental blame reconstruction (equivalently, the machinery behind code half-life analyses): replay the diff stream, assign every added line a birth, and record its death when a later diff deletes or replaces it. The output is a per-line record $\langle \text{repo}, \text{path}, \text{birth}, \text{death}, S, E, \text{author_type} \rangle$.

2.2 From lines to shapes

Line survival is high-variance: reformatting, renaming, and mechanical edits kill lines without killing *ideas*. The load-bearing signal is the survival of *shapes*—structural units recovered from the abstract syntax tree: function signatures, type/class definitions, module and package boundaries, public interfaces, and recurring design patterns. A shape σ is identified across commits by a stable structural key (e.g. a normalized signature or an interface hash) rather than by text, so it can survive an internal rewrite that replaces every one of its lines. We then measure $S(\sigma)$ exactly as above.

The empirical claim is that the distribution of $S(\sigma)$ is heavy-tailed and *informative*: a small fraction of shapes—clean abstractions, well-chosen interfaces—persist across the entire visible history, while the majority churn. Training a generator to prefer the long-lived tail is training it toward durable architecture, which is precisely the competence that cross-sectional corpora fail to teach.

2.3 Durability label

Given a survival distribution we define a per-unit durability label by upper and lower quantiles q_{hi}, q_{lo} of S (computed within-repository and within-language to control for base rates):

$$\ell(u) = \begin{cases} \text{DURABLE} & S(u) \geq q_{hi} \\ \text{EPHEMERAL} & S(u) \leq q_{lo} \\ \text{NEUTRAL} & \text{otherwise.} \end{cases}$$

EPHEMERAL units are not discarded—they are the negative class, and they are as valuable as the positives.

3 The three co-registered layers

The corpus fuses three sources, each answering a different question, joined on a common (*repo*, *commit*, *timestamp*) key.

Layer	Source	Teaches
HOW (reasoning + tools)	Claude Code session transcripts (~5,190 sessions, ~4.3 GB)	the agent trajectory: task $\rightarrow$ think $\rightarrow$ tool call $\rightarrow$ observe
WHAT (verified change)	file-history + git diffs	the actual before $\rightarrow$ after code
RESULT (label + reward)	survival + CI + human/ai tag	did it last? did it pass? who wrote it?

Table 1: The three data layers. Individually useful; jointly, they yield grounded (*instruction* $\rightarrow$ *trajectory* $\rightarrow$ *diff* $\rightarrow$ *outcome*) tuples.

The join is the value. Public agentic datasets have the HOW or the WHAT; almost none carry the RESULT at the granularity of survival, and none is additionally labeled by author type. Because our transcripts and commits are co-located on the same machines, each agent trajectory can be linked to the commit(s) it produced (by timestamp proximity, repository, and the version-control tool calls embedded in the trajectory), and each resulting hunk inherits a survival outcome. We can therefore ask, and learn from, the question no other corpus can pose: *which agent decisions produce code that lasts?*

3.1 The human/ai axis

Every commit carries an author-type label—HUMAN (the developer’s own hands, including historical corporate identities) or AI (an autonomous agent account). This axis is not decoration; it lets survival be conditioned on authorship, quantifying (for example) whether AI-authored abstractions survive at human rates, and isolating the trajectory features that predict durability regardless of author. In the reference dataset the AI account first appears in late 2024 and, within eighteen months, produces contributions at several times the HUMAN rate—so the corpus also documents, in real data, the transition of authorship from human to agent.

4 Construction pipeline

The pipeline is deliberately decomplexed—one directed path, one artifact per stage, one canonical implementation:

extract → **normalize** → **link** → **redact** → **format** → **version/store**.

Extract. Two collectors. (i) A *session* collector reads `~/ .claude/projects/*.jsonl` into structured trajectories. (ii) A *history* collector replays each repository’s git graph into the per-line/per-shape survival records of Section 2.

Fleet aggregation. Development is spread across a heterogeneous fleet of machines, each holding a partial `~/ .claude` and partial working copies. The session collector runs *per host* and ships its sanitized shard to a central store; shards are deduplicated by session id and content hash so re-runs are idempotent and no machine is a single point of truth.

Normalize. One canonical schema: `trajectory{task, turns[{think, tool_calls, tool_results}], outcome}` and `unit{repo, path, kind, birth, death, survival, edits, author_type, durable}`.

Link. Join trajectory ↔ commit ↔ survival on *(repo, commit, time)*.

Redact. Mandatory and non-negotiable. Session transcripts and git history are known to contain live credentials (private keys, cloud and service tokens). Redaction runs before any artifact leaves its host: secret scanning, key/token/BEGIN-block scrubbing, PII removal, and a private/public license gate. Failed or aborted sessions are *labeled*, not dropped.

Format. Emit SFT, preference, trajectory, and evaluation splits (Section 5).

Version/store. All artifacts and metadata land in a single versioned dataset (`zen-agentic-dataset`); large blobs are held out of git and backed up to a hub (HuggingFace), with a private repository for the full corpus and a public repository for the redacted, license-clean subset. The survival and authorship metadata form the provenance layer for every record.

5 Training and evaluation formulations

Survival is not merely a filter; it is an objective.

Supervised (durable targets). Standard next-token / next-action loss, with examples weighted by $w(u) = f(S(u))$ so the model imitates long-lived code more strongly than churn.

Preference / DPO. Pairs (DURABLE, EPHEMERAL) drawn from the same file and neighborhood give a natural preference signal: *prefer the change that would have survived*.

Outcome-verified RL. The reward is the join of *did it pass* (CI/test status) and *did it last* (survival), yielding a dense, non-gameable signal for policies that act over tool-use trajectories.

Trajectory modeling. Train directly on state transitions $s_t \rightarrow s_{t+1}$ (how a file evolves), teaching refactoring and growth rather than one-shot synthesis.

Survival-holdout evaluation. A benchmark that scores a model on whether its generated change matches (or would plausibly survive as well as) the change that historically endured at that point in the file’s trajectory. Unlike single-test pass/fail, this rewards durable design, not merely green pipelines.

6 Why survival-weighting produces better agents

Imitation learning on cross-sectional code teaches a model to reproduce the average of what was written—including the large fraction that was immediately undone. Outcome-verified benchmarks improve on this but reward only the narrow target of passing a fixed test, which is gameable and silent about architecture. Survival-weighting adds the missing axis: it rewards code that continued to be *correct enough, clear enough, and well-factored enough that future work built on it rather than around it*. That is the closest available analogue to a natural-selection objective for software, and it is obtainable at the scale of a career’s version-control history rather than the scale of hand-written rubrics. Because the same corpus carries the agent’s reasoning and the author-type label, it uniquely supports the question that matters for building coding agents: *which patterns of agent behavior yield durable code?*

7 Privacy, provenance, and ethics

The corpus is derived from private development history and therefore carries real obligations. Secret redaction is a hard gate, enforced before egress and verified in tests. Private-repository code is segregated under the private dataset with license controls; only redacted, license-clean records enter the public split. Every record retains provenance—source host, session id, commit, author type, and pipeline version—so any datum can be traced, re-derived, or withdrawn. The methodology stores *all* exported data and metadata under one versioned home with fleet-wide backup, so the corpus is durable, auditable, and reproducible.

8 Conclusion

The history of code is a longitudinal experiment that has already been run, at enormous scale, with time as the annotator. Reading survival as a label—and elevating the unit of interest from lines to the shapes that last—turns a developer’s version-control graph into a uniquely valuable training corpus for agentic software models: one that teaches not what code was written, but what code deserved to endure. Co-registered with agent trajectories and an author-type axis, it lets us train and measure agents against the only judge that ever mattered, which is the future.